

Chapter 1

A formal methodology to specify hierarchical agent-based systems

Carlos Molinero, César Andrés and Manuel Núñez

Dept. Sistemas Informáticos y Computación

Facultad de Informática

Universidad Complutense de Madrid, 28040 Madrid, Spain

e-mail: {molinero, c.andres}@fdi.ucm.es, mn@sip.ucm.es

In this paper we introduce a formal framework to specify agent-based systems in which each of the agents is specialized in a single task that will be fulfilled by making calls to other *simpler* agents. In other words, we are interested in systems that can perform a task by subdividing it in easier tasks and by using the knowledge about each agent already introduced in the system. The idea is to prefabricate a basic structure that can be reused by either changing the main goal or by adding several different specialized agents.

The main characteristic of our methodology is that the each complex agent contains a schematic definition of other agents. Each agent is thus able to retain and produce certain information such as the time needed to accomplish a certain task taking into account a given set of agents and resources. This allows to quickly access information regarding the necessities in resources and derive the demands to other subsystems.

In order to increase the applicability of our approach, we have fully implemented a tool that allows us to graphically specify complex systems. In addition, the tool allows us to simulate the behavior of the specified systems so that some interesting properties, such as starvation and maximal progress, can be studied.

1.1. Introduction

The representation and study of communities where intelligent (electronic) agents replace their (human) owners is a topic that has attracted a lot of interest. In particular, there is on-going research in technologies able to model users by means of agents which autonomously perform electronic transactions (see¹ for a survey on the topic). In order to increase the power of these agents they must know the preferences of the corresponding user. In this line, the concept of *utility function* is very useful. Essentially, a *utility function* returns a real number for each possible basket of goods: The bigger this number is, the happier the owner is with this basket. Intuitively, agents should appropriately simulate the systems that they are representing by considering the utility function that would establish the expected behavior (see e.g.²⁻⁷). In fact, there exists several proposals showing how agents can be trained to learn the preferences of users (see e.g.^{4,8,9}). Besides, a formal definition of the preferences of the user provides the agent with some negotiation capacity when interacting

with other agents.^{9–11} Let us remark that, in most cases, utility functions take a very simple form. For instance, they may indicate that an agent A is willing to exchange the item a by the items b and c .

The motivation of this work is to be able to create a flexible computational system capable of offering different solutions to new problems. Instead of pre-introducing thoroughly all the possible behaviors of a system, the system reorganizes its structure to be able to cope with new petitions coming from the user. It bases itself in a series of behaviors (or task) that will be called *atomic* tasks which are the main bricks upon which to construct any new behavioral answer to the new needs that may appear.

Even though there are general purpose formalisms to formally describe complex concurrent systems (such as process algebras^{12–14} or Petri Nets^{15,16}) they are not suitable to describe agents since these languages and notations do not provide specific operators to deal with the inherent characteristics of agents. However, there has been already several studies to formally describe the use of intelligent electronic agents that are nested into one another (see, for example,^{17,18} for two approaches based on Petri Nets and automata,^{19,20} for approaches based on process algebras, and^{21,22} for approaches based on finite state machines). Most of these approaches have been created in favor of comprehensibility. Therefore they facilitate to derive and apprehend new properties. Moreover, these approaches have also shown that the recursive definition of agent-based systems allows to keep the original properties of basic agents.

If we try to incorporate the base of facts to a system, there will always be a lack of capacity to implement every possible structure of the agent, every different solution to the same problem, and every combination of small pieces that constitute a complex problem. That is why we think that there is a need for a new framework to formally define the class of systems previously described. We believe it is easier and more feasible to incorporate bits of knowledge by having the system recomposing this information into complex tasks. This approach simplifies at least two aspects. First, it helps to ensure the completion of the base of facts. Second, it allows to relocate the different agents, due to its modularity, so that they can be spread over a network to parallelize some of the tasks.

The main skeleton of the system is this so called *atomic* agents which are the ones that are in charge of executing the actual transformations that will occur in the system. Through the recombination of this *atomic* agents, new and *complex* agents are created. The first advantage of this approach, is that once the *atomic* agents are defined little interaction from part of the user is needed, as well, the lack of programming abilities will also not be a disadvantage. All is needed is to program the underlying specific characteristics of the system, that is what this system will be able to handle, by example procedures to call to the motors of a robot, and whatever future need of the user will be handled by the system itself.

If we take a look in another direction, we would like to assimilate the systems that we are interested in to a *common places* structure in which one is able to locate the rest of the structure from higher order points. If we use the subway lines as a metaphor, we only need to know the location of the different stations, but the exact location of that small fruit

shop that we are trying to reach is bounded to the location of the closest metro station. Once we arrive to that particular metro station, we will check the neighborhood map so that we can find the shop; we do not need to know in advance all the local maps associated with all the stations of the network. That is how our systems will work: Once we have all the atomic agents, each time that a new complex agent, embracing the knowledge of several atomic agents, is created we will refer to this new agent when making subsequent calls to the system. In this line, we are able to *forget* how atomic actions are performed because we have a higher order element to which we can call upon. In any case, even with a complex structure, atomic agents are still the ones that execute *real* tasks. Using another metaphor we could say that our systems are similar to economic structures in which there exist intermediate agents that gives us the result of the transformation of resources as a final product. These agents, in a hidden way, contract the prime manufacturers that create these resource transformations.

Another point in favor of our approach is that it allows us to have an unbounded growth (equivalently, subdivisions as small as needed) either by adding systems in between existing ones or by assigning new atomic agents to the ones that we had before.

In order to cope with time and calculation limitations, the system is subdivided into a hierarchical structure in which agents are ordered considering its complexity (how many agents does it uses to perform a task) and also the “field” of knowledge to which that specific agent is inserted into. I assume here, that if agents are located correctly into groups of related matters, meaning by it that if two agents perform similar tasks, they will be inserted together, when the pieces are recombined following the procedure explain later in this texts, they will be kept close in the hierarchical tree.

This hierarchical structure is implemented through a system of so called *cells* that conglomerate similar agents together. This cells are ordered in a tree structure, which all hang from the main cell, every petition will be inserted into the main cell and will be carried on down on the tree until it has been processed. This allows too, although it has not been addressed in the implementation, the possibility of facilitating a computational limit, meaning that the search could stop after it has descended a limit of levels down in the tree structure, if the system needs to take decisions in a certain amount of time that can not be surpassed.

It is important to note that the way our systems are subdivided, in so called *communication cellules*, facilitates their deployment in a distributed system in which one can obtain a perspective of variable magnitude of the global tasks. This holds as long as we keep the hierarchical structure of the ensemble.

In this work, agents are treated as knowledge elements, agents are inserted as new abilities that the system can have access to, and then agents are created through the recombination of these abilities to generate complex tasks. Each agent represents a knowledge, each action is therefore identified with a knowledge about that action. When a petition is made to the system, it tries to accomplish it by reusing its bits of knowledge. In comparison with the human behavior, this is thought as a metaphor of the “mirror neurons”, discovered by chance by the group formed by Giacomo Rizzolatti , Giuseppe Di Pellegrino, Luciano Fadiga, Leonardo Fogassi, and Vittorio Gallese at the university of Parma, Italy, while mea-

surging the activities of neurons regarding the movement of a monkey. Unexpectedly some neurons fired not only when the monkey was moving the hand, but also when it looked how someone else was moving the hand. These led to a new theory of learning in which it is stated that they way humans and other mammals learn is through the neural mimic of activities seen in other individuals. Therefore the how-to knowledge and the actual act is primarily fired by the same neurons.

The system apprehends more complex concepts in the same way children do, first as a baby you start to involuntary move your muscles, noticing that that makes your arm move, with time you learn to control your arm, and the modifications that it performs in your surroundings, afterwards, as more concepts and experiences (in our system petitions) are incorporated, you no longer think about moving the arm but of reaching an object and grabbing it. That is, once we know how to perform an action the underlying mechanisms are automated, we no longer have to think about them.

The generic task method states that the structure and representation of domain knowledge is completely determined by its use (much as it happens in our approach, where concepts are only derived from the actual task that can be accomplished by atomic agents). The main problem underlying generic tasks is related to the predetermined problem solving strategy that they use (we are able to overcome this situation, since our methodology proposes a flexible problem solving strategy, that will vary depending on the kind of atomic agents included in the system).

The rest of the paper is organized as follows. In Section 1.2 we will discuss some of the most relevant papers in the field of agents. In Section 1.3 we introduce some auxiliary notation that will be used during the rest of the paper. In Section 1.4 we present the formalism to describe our systems. In Section 1.5 we briefly present our tool and give a small example to show its main features. Finally, in Section 1.6 we present our conclusions and some lines for future work.

1.2. Overview of some relevant papers in the field of “agents”

1.2.1. *Pattie Maes - The dynamics of action selection*

This paper (²³) addresses the problem of choosing an action in an autonomous multi-agent system. Actions are chosen following a rational goal oriented fashion, but this approach can have conflicting goals, it should be adaptive to new situations, and there exist the possibility of a certain component failing, making it harder to reach the final goal.

This is done so in the situation of a mindless multi-agent system, such as those of Brooks’ subsumption architectures. This systems, although desirable for properties such as modularity, distributed behavior, flexibility and robustness lack of a proper action selection procedure: which agent should become active? and what are the factors that determine a cooperation among certain agents?. The hypothesis presented in this paper are that rational action of the global system can emerge, and that there is not the need for “bureaucratic” agents (agents that decide which agent should become active).

There exist in this framework several parameters to be tuned by the user, allowing to

have different kind of action selection procedures, such as more/less data oriented, goal oriented, deliberated, fast.

Agents make part of a hierarchical system in the way that the activation of an agent is linked in a network of predecessor and successor links, which describe what agents should be activated before the current agent that is trying to perform an action. An agent is described by the tuple (l_p, l_a, l_d, a) where:

- l_p is a list of preconditions which have to be fulfilled before the agent can become active.
- l_a and l_d represent the post-conditions in terms of a add list and delete list scheme.
- a is the level of activation of the agent.

The links of the network are used to spread activation among agents belonging to it. When an agent's preconditions are true it spreads part of its activation level to its successors otherwise it augments the activation level of its predecessors. The algorithm that takes place at every time step is composed of the following steps:

- (1) The input from the state and goals to an agent is computed.
- (2) The spreading of activation of an agent is computed.
- (3) A locally computed "forgetting" factor ensures that the overall activation level remains constant.
- (4) The agents which fulfills the following 3 conditions becomes active: it has to be executable, its level of activation has to surpass a certain threshold and it must have a higher activation level than all other agents which fulfills the preconditions.

The parameters to be tuned in the system are:

- The threshold for becoming active.
- The percentage of their activation that is spread forward to other agents.
- The percentage of their activation that is spread backward to other agents.
- The relative amount of external input that comes from the goals as opposed to from the state of the environment.

1.2.2. Yoav Shoham - Agent-oriented programming

In his paper "Agent-oriented programming", Yoav Shoham (cite²⁴) proposes a new computational framework that promotes a societal view of computation, in which agents are combined to perform a certain computation. Agents are defined by their mental state, which is decomposed into beliefs, decisions, capabilities and obligations.

For Shoham an agent is:

an entity whose state is viewed as consisting of mental components such as beliefs, capabilities, choices and commitments. These components are defined in a precise fashion and stand in rough correspondence to their common sense counterparts.

Clearly it is a definition of the term agent biased towards its own work, since he is going to define precisely those elements as part of his agent programming language. Since the possibility of ascribing beliefs, decisions, capabilities and obligations to any element, even the most simple of the systems, and therefore following its definition, considering that anything can be said to be an agent, he uses the words of John McCarthy, to explain when ascribing those mental states to a system is something useful:

“To ascribe beliefs, free will, intentions, consciousness, abilities, or wants to a machine is legitimate when such an ascription expresses the same information about the machine that it expresses about a person, It is useful when the ascription helps us understand the structure of the machine, its past or future behavior, or how to repair or improve it. It is perhaps never logically required even for humans, but expressing reasonably briefly what is actually known about the state of the machine in a particular situation may require mental qualities or qualities isomorphic to them. Theories of belief, knowledge and wanting can be constructed for machines in a simpler setting than for humans, and later applied to humans. Ascription of mental qualities is most straightforward for machines of known structure such as thermostats and computer operating systems, but it is most useful when applied to entities whose structure is incompletely known.”

Shoham adopts the S5 modal logic which have properties that includes tautological closure, positive introspection and negative introspection. And the semantics adopted are the possible world semantics.

He states that decisions are logically constrained though not determined by the agent's beliefs. These beliefs refer to the state of the world, to the mental state of other agents and to the capabilities of this and other agents. This perspective motivates the introduction of two mental categories: *belief* and *decision* (or choice), and another not mental per se construct which is *capability*. Although decision will be treated in terms of *obligation* as an obligation to oneself.

Definitions of the mental categories:

- Time, all operators are related to time.
- Belief, an agent a believes something on a certain time t : $B_a^t \phi$
- Obligation, agent a has an obligation to agent b on time t : $OBL_{a,b}^t \phi$
- Decision, agent a obliges itself: $DEC_a^t \phi = OBL_{a,a}^t \phi$
- Capability, agent a is capable of doing something at time t : $CAN_a^t \phi$

All of these constructs maintain a certain set of properties, like internal consistency of beliefs and obligations, good faith (agents only commit to what they believe themselves capable of), introspection (agents are aware of their obligations) and persistence of the mental state (agents have perfect memory of beliefs and obligations, and they only let go of a belief if they learn a contradictory fact).

Later on in the paper he discusses the AGENT0, a language made to create agents and define its mental categories, and the message passing (communications) in between the agents, also discusses the need for agentification (create agent-like representation out of cameras or other devices so they can be used by agents).

1.2.3. Rodney A. Brooks - *Elephants don't play chess*

Rodney A. Brooks have written a series of papers talking about the subsumption architecture one of them is “Elephants don't play chess” which is kind of a summary of all the developments made by his team, and a theoretical comparison with symbolic approaches.

He adheres himself to a current dogma in the AI world, called *situated activity*, based on the *physical grounding hypothesis*. Which states that intelligence is a general property that can arise from the combination of a series of different reactive and situated (meaning that they only work located in a real world) agents that handle different parts of the overall robot's behavior.

His thesis against the symbolic approaches is that they are too field dependent, they are not capable of adapting to the noise existing in the real world, and that the sensory equipment is incapable of presenting with accurate symbolic descriptions of the objects that constitute the real world, rendering such approaches practically unusable. Also the number of calculations necessary to find solutions in the search spaces (usually for NP-complete spaces), constitute another drawback of trying to use the symbolic position.

The physical grounding hypothesis works based on the assumption that every system needs to have its representations grounded in the physical world. Therefore to connect the system by sensors and actuators to the real world is the primary interest of these approaches. This kind of approach forces the construction of the system in a bottom up manner, everything has to be concrete responses to the environment.

Another important point in the paper is the explanation of the subsumption architecture. It is a way to program the robot based on incremental layers, each one of them connecting perception to action, based on augmented finite state machines (AFSM). All of this layers are compiled to simulate parallelism. There are two subsumption languages, the old and the new, used in different experimental robots. I will hereby quote the general definition for the old subsumption language:

Each augmented finite state machine has a set of registers and a set of timers, or alarm clocks, connected to a conventional finite state machine which can control a combinational network fed by the registers. Registers can be written by attaching input wires to them, and sending messages from other machines. The messages get written into the registers by replacing any existing contents. The arrival of a message, or the expiration of a timer, can trigger a change of state in the interior finite state machine. Finite state machine states can either wait on some event, conditionally dispatch to one of two other states based on some combinational predicate on the registers, or compute a combinational function of the registers directing the result either back to one of the registers or to an output of the augmented finite state machine. Some AFSMs connect directly to robot hardware. Sensors deposit their values in certain registers, and certain outputs direct commands to actuators.

A series of layers of such machines can be augmented by adding new machines and connecting them into the existing network in a number of ways. (...)

The new subsumption language, uses behaviors (that are in fact AFSMs). The main tools to interact in between the behaviors are message passing, suppression and inhibition. Another difference with the old language is that behaviors can share registers, and a new

more general timing mechanism than the original alarm clocks.

Later on the article explains one by one the different robots build with this subsumption architecture. Showing how different reaction layers can make emerge an “intelligent” behavior.

1.3. Preliminaries

In this section we introduce some notation that will be used throughout the rest of the paper.

Since users have different preferences, in order to properly design agents the first step consists in expressing these preferences. In order to extract preferences from users several mechanisms have been presented in the literature (see e.g.^{4,25,26}). In this paper, preferences in a given moment will be given by a *utility function*. These functions associate a value (a utility measure) with each possible combination of resources a user could own. Alternatively, other mechanisms such as *preference relations* could be used (see e.g.²⁷ for conditions to transform one of the possibilities into the other).

Definition 1.1. Let $\mathbf{R}_+ = \{x \in \mathbf{R} \mid x \geq 0\}$. Vectors in $\mathbf{R}_+^n$ (for $n \geq 2$) are usually denoted by $\bar{x}, \bar{y}, \dots$. The vector 0 denotes the tuple having all the components equal to zero. Given $\bar{x} \in \mathbf{R}_+^n$, x_i denotes its i -th component. Let $\bar{x}, \bar{y} \in \mathbf{R}_+^n$: $\bar{x} \leq \bar{y}$ if for all $1 \leq i \leq n$, $x_i \leq y_i$ holds.

If there exist n different kinds of resources then a *utility function* is any function $f : \mathbf{R}_+^n \mapsto \mathbf{R}_+$. $\square$

Intuitively, $f(\bar{x}) > f(\bar{y})$ means that $\bar{x}$ is preferred to $\bar{y}$. For instance, if the resource x_1 denotes the amount of apples and x_2 denotes the amount of oranges, then $f(x_1, x_2) = 3 \cdot x_1 + 2 \cdot x_2$ means that, for example, the agent is equally happy owning 6 apples or 9 oranges. Let us consider another agent whose utility function is $f(x_1, x_2) = x_1 + 2 \cdot x_2$. Then, both agents can make a deal if the first one gives 3 oranges in exchange of 4 apples: After the exchange both are happier. Alternatively, if x_2 represents the amount of money in any currency (for example in dollars) then the first agent would be a customer while the second one might be a vendor. A usual assumption is that no resource is a *bad*, that is, if the amount of a resource is increased, so does the value returned by the utility function. Using a derivative expression, this property can be formally expressed as $\frac{\Delta f(x_1, \dots, x_n)}{\Delta x_i} \geq 0$ for any $x_1, \dots, x_n \in \mathbf{R}$ and $1 \leq i \leq n$. This requirement does not constrain the expressive power of utility functions, as the existence of any undesirable resource can be always expressed by considering a resource representing the *absence* of it.

During the rest of the paper we consider that agents use messages to communicate among them. The next definition introduces the different kinds of messages that can be sent.

Definition 1.2. We consider seven different types of messages and we enumerate them as follows:

TYPE	CONTENT
1	Information
2	Negotiation
3	Proposal message
4	Acceptance message
5	Hiring message
6	Job started message
7	Job finished message

Let ID be a set of agent identifiers. A *message* is a tuple (T, s, d, c) , where T is the *type* of message, $s \in ID$ is the agent source of the message, $d \in ID \cup \{\star\}$ is the agent destination of the message ($\star$ represents a broadcast message) and $c \in String$ represents the string of characters containing the message.

We denote by λ the empty message. We denote by $\mathcal{M}$ the set of all messages. $\square$

Next we will define a collection of identifiers to be able to univocally address agents and cellules.

Definition 1.3. Let w be a system (see Definition 1.8). The set of all possible systems is represented by $\mathcal{W}$. We denote by ID^C the set of cellule identifiers that are assigned uniquely to each of the cellules. The function **newIdCellule** : $\mathcal{W} \rightarrow ID^C$ returns an unused identifier for the world w . We use a special identifier $null \in ID^C$ to denote an empty cellule. We denote by ID^A the set of agent identifiers that are assigned uniquely to each of the agents belonging to the system. The function **newIdAgent** : $\mathcal{W} \rightarrow ID^A$ returns an unused identifier for an agent. $\square$

1.4. Definition of the formalism

In this section we present our formal language to specify complete systems as well as all the agents belonging to them. The main idea consists in having a *world* that is composed of *communication cellules*. These components will be interconnected in a hierarchical way, that is, the main communication cellule will hold the main agent while each of the next communication cellules will hold simpler agents. This process is iterated until we reach the last level that will hold only atomic agents.

We will make a distinction between *generic agents*, in other words, agents fulfilling a complex task (which will be done by calling other agents) and *atomic agents*, which are simple agents in charge of executing the actual transition. This distinction is merely represented as a different set of variables taken from the same general definition of an agent. Agents will send messages by using the communication cellules. These cellules will forward these messages to other cellules that will broadcast them to the agents under their control, until the final atomic agent is reached. Then, an atomic portion of the global goal will be produced through the transformation of resources that this atomic agent fulfills.

Each agent will have a different utility function, as defined in the previous section, depending on the *utility* that each of the resources represent for the specific agent. This function will take into account different combinations of resources to decide which transition should be used.

We will start by defining the simpler element in the system, the agents, and scale up in complexity to define the complete system.

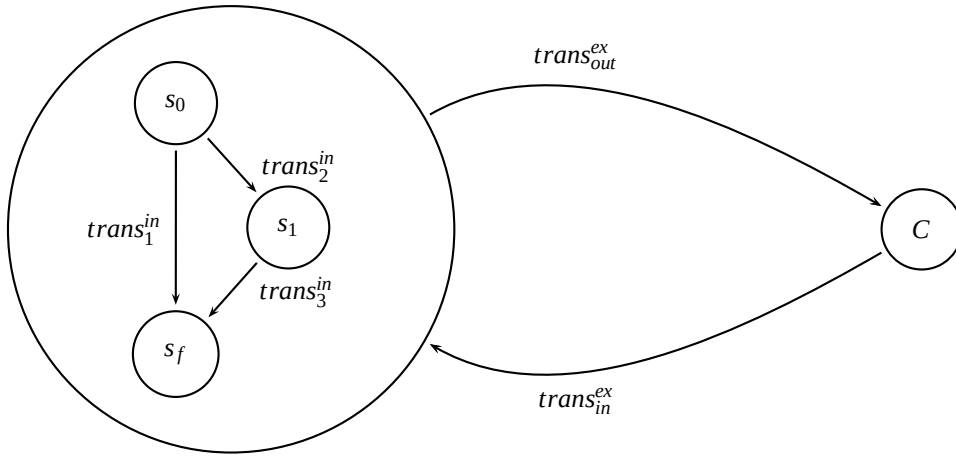


Fig. 1.1. A generic agent.

Definition 1.4. We denote by $\mathcal{A}$ the agent domain; $a_1, \dots, a_n$ will be used to denote elements of $\mathcal{A}$.

An *agent* is a tuple:

$(id, S, s_0, s_f, R, \mathcal{V}, T_{in}, T_{ex}, ib, ob)$ where:

- $id \in ID^a$ is the agent identifier.
- S is the set of states.
- $s_0 \in S$ is the initial state.
- $s_f \in S$ is the final state or *goal* state.
- R is the set of resources of the agent.
- If $|R| = m$ then we can use *valuations* $\rho : R \mapsto \mathbb{R}_+^m$ that return the current value of each variable. We denote by $\text{Val}(R)$ the set of all valuations of R . $\mathcal{V} \in \text{Val}(R)$ is the initial valuation of the agent.
- $T_{in} \subseteq S \times ID^a \times \mathbb{R}_+ \times S \times \mathcal{M} \times \wp(\text{Val}(R)) \times \text{Val}(R)$ is the set of internal transitions. The set of internal transitions must fulfill the following restrictions:
 - if $|T_{in}| = 1$ then $\forall (s_1, id', t, s_2, m, D, Z) \in T_{in} : id' = id$.
 - if $|T_{in}| > 1$ then $\forall (s_1, id', t, s_2, m, D, Z) \in T_{in} : id' \neq id$.
 - $|T_{in}| \geq 1$.
- $T_{ex} \subseteq S \times \mathcal{M} \times \mathcal{M} \times S \times \mathcal{F}(\text{Val}(R) \mapsto \text{Val}(R))$ is the set of internal transitions,

where $\mathcal{F}(\text{Val}(R) \mapsto \text{Val}(R))$ denotes the set of functions over $\text{Val}(R)$. We say that an agent is well formed if for all $s \in S$ and all $m_i \in \mathcal{M}$ there exist m_o, Z, s_2 such that $(s, m_i, m_o, s_2, Z) \in T_{ex}$.

- $ib, ob \in \wp(\mathcal{M})$ are the input and the output buffers, respectively. We will use them to receive incoming messages (ib) and to send outgoing messages (ob). We can use the functions Choose : $\wp(\mathcal{M}) \mapsto \mathcal{M}$, Remove : $\wp(\mathcal{M}) \times \mathcal{M} \mapsto \wp(\mathcal{M})$ and Concat : $\mathcal{M} \times \wp(\mathcal{M}) \mapsto \wp(\mathcal{M})$, having the usual meaning, to manage buffers.

□

Intuitively, an internal transition $(s_1, id', t, s_2, m, D, Z) \in T_{in}$ is a tuple where s_1 is the initial state, s_2 is the final state, id' identifies the agent assuming the transition^a, $t \in \mathbb{R}_+$ is the time^b consumed to perform the transition, m represents the output message that the agent emits to the outgoing buffer (ob) when this transition is triggered (the empty message is denoted by λ), $D \subseteq \text{Val}(R)$ denotes a subset over the set of valuations that denotes which valuations allow the transition to occur, and $Z : \text{Val}(R) \mapsto \text{Val}(R)$ is the transformation of resources produced by this transition. We write $s_1 \xrightarrow{id', m, D, Z}_{ex} s_2$ as a shorthand of $(s_1, id', s_2, m, D, Z) \in T_{in}$.

External transitions are tuples (s_1, m_i, m_o, s_2, Z) where s_1 is the initial state, s_2 is the final state, m_i is the incoming message, m_o is the outgoing message, and $Z : \text{Val}(R) \mapsto \text{Val}(R)$ is the transformation of the values in this transition. We write $s_1 \xrightarrow{m_i/m_o, Z}_{in} s_2$ as a shorthand of $(s_1, m_i, m_o, s_2, Z) \in T_{ex}$.

In Figure 1.1 we show a graphical representation of a generic agent. Next, we introduce some auxiliary concepts that will be useful to describe the evolution of agents.

Definition 1.5. An agent is called atomic if it is in charge of executing a single task. Formally, we use a function $\text{atomic} : \mathcal{A} \mapsto \text{Bool}$, such that for all $a \in \mathcal{A}$ we have $\text{atomic}(a) = (|T_{in}| = 1)$.

In order to have the current state of an agent we specify its configuration as an element belonging to $S \times \text{Val}(R) \times \wp(\mathcal{M}) \times \wp(\mathcal{M})$. Configurations are modified through the performance of either internal or external steps:

Internal step of the system:

Given a configuration $M = (s, \mathcal{V}, ib, ob)$, an internal transition $s_1 \xrightarrow{id', t, m, D, Z}_{in} s_2$ will be triggered if $D(\mathcal{V})$ and will modify the configuration to $(s_2, Z(\mathcal{V}), ib, \text{Concat}(m, ob))$.

External step of the system:

Given a configuration $M = (s, \mathcal{V}, ib, ob)$, an external transition $s_1 \xrightarrow{m_1/m_2, Z}_{ex} s_2$ will be triggered if $m_1 = \text{Choose}(ib)$ and will modify the configuration to $(s_2, Z(\mathcal{V}), \text{Remove}(ib, m_1), \text{Concat}(m_2, ob))$. □

^aIf $id = id'$ then we are considering an atomic agent that is itself in charge of its transition

^bThis value will be defined by default only for atomic agents since complex agents will calculate the associated time from the information collected from the contracts with other agents. In addition to consider the sum of all the involved time values, we have to add a certain delay to simulate the time that it takes to perform communications among agents.

In order to assign each agent to a specific communication cellule, so that the hierarchical structure can be kept, we will define the traces of an agent. The set of all traces that an agent can perform will be use to compare agents.

Definition 1.6. Given an agent, we define a trace as a possibly empty sequence of outgoing messages $tr = \langle m_1, \dots, m_n \rangle$ with $n \geq 0$, where each m_i is in $\mathcal{M}$. If $n = 0$ then we have the empty trace $\langle \rangle$.

Let $a = (id, S, s_0, s_f, R, \mathcal{V}, T_{in}, T_{ex}, ib, ob, \Gamma)$ be an agent. We say that a (possibly empty) sequence of messages $\langle m_1, \dots, m_n \rangle$, with $n \geq 0$, is a trace of a if there exists a sequence of transitions belonging to T_{in} and starting at s_0 such that

$$s_0 \xrightarrow{id, m_1, D, Z} \mathcal{A}, in s_1 \dots s_{n-1} \xrightarrow{id, m_n, D, Z} \mathcal{A}, in s_n$$

We denote by $\text{tr}_{\mathcal{A}}(a)$ the set of all traces of a .

Given two agents $a_1, a_2 \in \mathcal{A}$ we say that a_1 is more powerful than a_2 , denoted by $a_2 \leq_p a_1$, if $\text{tr}_{\mathcal{A}}(a_2) \subseteq \text{tr}_{\mathcal{A}}(a_1)$. $\square$

Cellules are elements that serve as baskets of agents to reunite, organize, conglomerate and handle petitions as well as calls to the agents.

Definition 1.7. A *cellule* is a tuple $(\mathcal{A}, id, \text{Sons}, \text{Father}, ib)$ where

- $\mathcal{A} \subseteq \text{ID}^A$ is the set of agents that belong to the cellule.
- $id \in \text{ID}^C$ is a unique identifier for this cellule.
- $\text{Sons} \subseteq \text{ID}^C$ is the set of identifiers of the sons of this cellule. If $\text{Sons} = \emptyset$ then we are in a node cellule.
- $\text{Father} \in \text{ID}^C$ is the identifier of the cellule that is father of this cellule. If $\text{Father} = \text{null}$ then we are in the initial cellule, from which all other cellules are defined.
- $ib \subseteq \mathcal{M}$ is the input buffer where messages will be stored.

We denote by $\mathcal{C}$ the set of all cellules. $\square$

Let us remark that when an agent is added to a cellule, the incoming buffer of the agent is connected to the outgoing buffer of the cellule, while the outgoing buffer of the agent is associated with the input buffer of the cellule. This means that an agent does not send messages to another agent; it sends them to the associated cellule. Similarly, an agent receives messages only from the cellule to which it is attached. A graphical representation of this can be seen in Figure 1.2 where we can observe several agents connected to the cellule C .

Next, we define the whole system that contains in a tree like structure implicitly defined by the father-son relationship, the cellules that conform the whole system.

Definition 1.8.

We say that our *system* (sometimes called *world*) is defined with a so called *origin cellule* from where the tree of cellules hang and by the vector of resources available in the

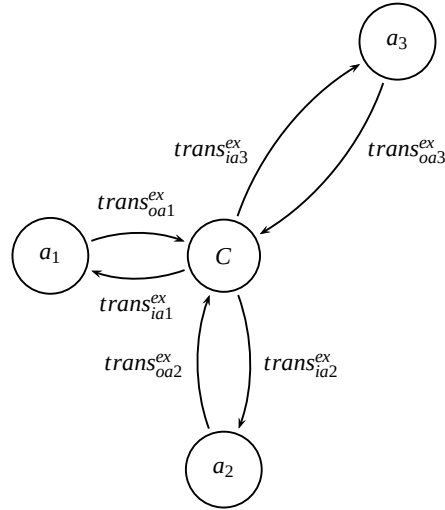


Fig. 1.2. Communication cell.

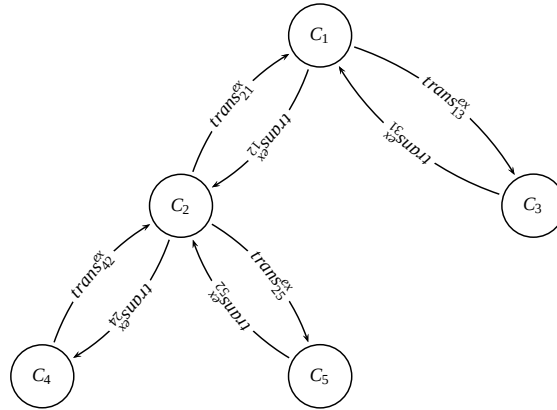


Fig. 1.3. World.

system. Therefore, a *system* is a pair $w = (c, \bar{x})$ where $c \in ID^C$ is the origin cellule, and $\bar{x}$ is the set of resources with which we deal in this world $\bar{x} \in \mathbf{R}^n$.

We will use a running example to illustrate previously introduced concepts. In order to ease the presentation, we have simplified the real system that we have represented in our formalism. $\square$

All agents that are not *atomic* are *complex*, there are two ways to create agents one is to insert an atomic agent during the creation of the system and the other is through *petitions* to the system, being the system in charge of recombining atomic and/or complex agents

already embedded in the system to create a new complex agent.

Definition 1.9. We say that a *petition* is a tuple $pet = (f^u, \bar{y}, \bar{o})$ where $f^u \in \mathcal{F}$ is a utility function, $\bar{y} \in \mathbb{R}^n$ is the vector of resources that is added to the resources already existing in the world, and $\bar{o} \in \mathbb{R}^n$ is the objective of the transitions, that is, the vector of resources that we expect to have after performing the petition. $\square$

Intuitively, if we have a petition $pet = (f^u, \bar{y}, \bar{o})$ is a petition, and $a = (id, ib, P)$ is the agent that has created the petition, if $\exists p \in TR$ such that exists $(p, id_p) \in P : VP(id_p) + \bar{x} + \bar{y} \geq \bar{o}$.

Example 1.1.

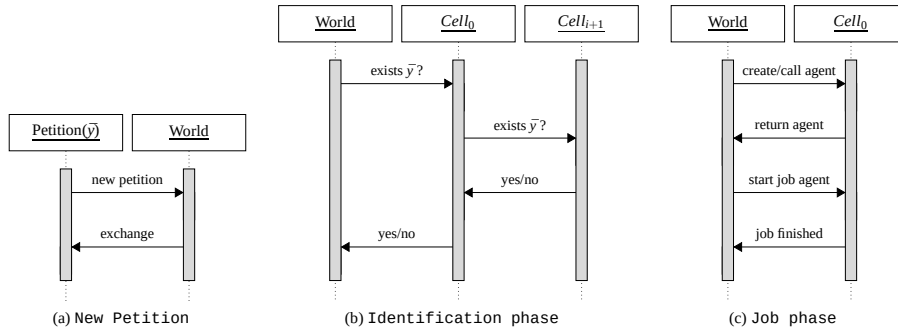


Fig. 1.4. Schematic diagrams of world behaviour.

We will explain the main messages by applying a petition (a graphical representation is given in Figure 1.4). Let us consider a petition $pet = (f^u, \bar{y}, \bar{o})$. The tuple has 3 elements, the first one is the initial resources, $\bar{y} = [1000, 500, 500, 100, 0, 100, 0, 0, 500]$, the second one a utility function (in this case $f^u = 10 \cdot x_1 + 5 \cdot x_9$), and the third element of the tuple is the objective tuple of resources $\bar{o} = [0, 0, 0, 0, 1, 0, 1, 0, 0]$.

The first diagram of Figure 1.4 denotes that $pet = (f^u, \bar{y}, \bar{o})$ is inserted in the world $w = (I, \bar{x})$. When a new petition is inserted in the world, the resources of the petition are added to the existing vector of resources. After this initial stage, the world “asks” to its structure of cellules if there are any agent(s) which can achieve the objective function $\bar{o}$. $\square$

1.5. The $\mathcal{A} \setminus \sqcup$ tool

In this section we briefly describe the $\mathcal{A} \setminus \sqcup$ tool. In the previous section we have presented a modular framework that has the capability of expressing constraints, specifications, agents, cellules, and resources. The $\mathcal{A} \setminus \sqcup$ tool, developed in Java, facilitates the definition of systems following our methodology so that a user of our methodology can abstract most of the mathematical technicalities needed to define a system/world. We illustrate the behavior of the most relevant phases of the framework by following an example

from scratch.^c First we will show how we can create a new world, assigning resources, agents, cellules, etc. Second, we will see the communication among agents, in order to obtain resources, rent other agents, etc. And finally we will see how we achieve the proposed goal.

In order to start the simulation, a preliminary phase is necessary to create the world, the cellules, and the agents. We can record the traces from a cellule and the preprocessing module treats the log file containing the observed trace in order to make a formatted trace. In other words, it can generate agent traces that follow the format established in our framework. We create the world called *CompLutense* and inside it we add three different Cellules (C_1, C_2, C_3) with some agents in them. In Figure 1.6 we can observe how $\mathcal{A} \setminus \sqcup$ shows the generated world as well as an agent of this world. The set of agents that we have introduced in the system is defined in Figure 1.5.

NAME	LEVEL	INITIAL	RESOURCES	GOAL
agent ₁	1	s ₁	(410, 290)	s ₆
agent ₂	1	s ₁	(370, 340)	s ₆
agent ₃	2	s ₁	(175, 151)	s ₃
agent ₄	2	s ₁	(220, 241)	s ₄
agent ₅	2	s ₁	(312, 287)	s ₅
agent ₆	2	s ₂	(149, 150)	s ₄
agent ₇	2	s ₂	(224, 159)	s ₅
agent ₈	2	s ₂	(220, 176)	s ₆
agent ₉	2	s ₃	(231, 148)	s ₆
agent ₁₀	2	s ₄	(149, 101)	s ₆
agent ₁₁	3	s ₁	(70, 100)	s ₂
agent ₁₂	3	s ₂	(75, 50)	s ₃
agent ₁₃	3	s ₃	(72, 68)	s ₄
agent ₁₄	3	s ₄	(72, 35)	s ₅
agent ₁₅	3	s ₅	(72, 43)	s ₆

Fig. 1.5. Agents in the world.

As we can observe in the table, agent₁₅ is an atomic agent because it is in the lower level of the cellules and, according to the definition of level, this cellule can only have atomic agents; an atomic actions only goes from s_n to s_{n+1} . The values that are in the fifth column represent the money and the time. Even though there are other resources involved in the system we have only represented these two resources since they are the most relevant for our example. *CompLutense* represents a world where agents can build houses. We have subdivided the task of *building* in six states. In Figure 1.7 we give a description of each of these states.

Once the world is created, we start with the second phase. We generate the connections that are in charge of telling the agent that it has reached a certain state or to allow the agent

^cEven though this is a toy example, so that we can concentrate on the main features of the tool, we have already tested our tool with some more complex examples. However, as we indicate in the last section of the paper, we still need to use our tool to specify a *real* system.

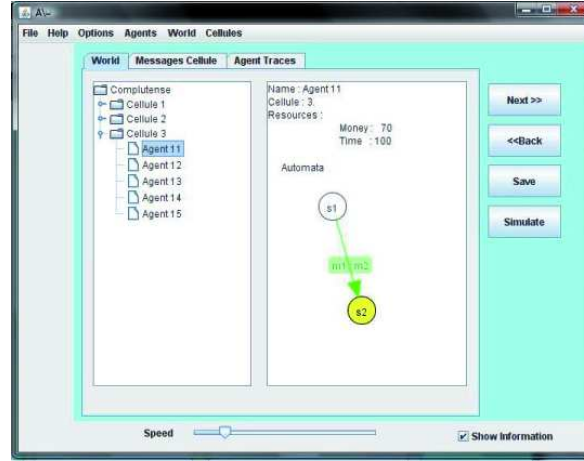
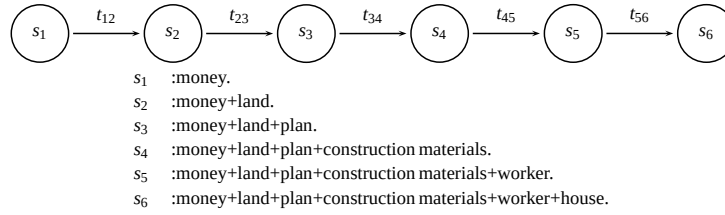
Fig. 1.6. Phase 1 in $\mathcal{A}/\mathcal{L}$.

Fig. 1.7. Global task.

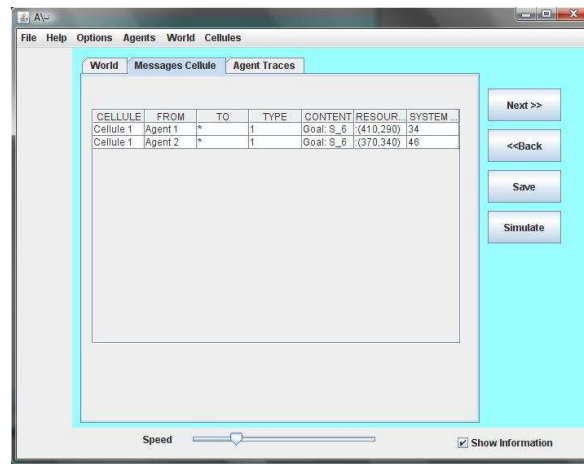
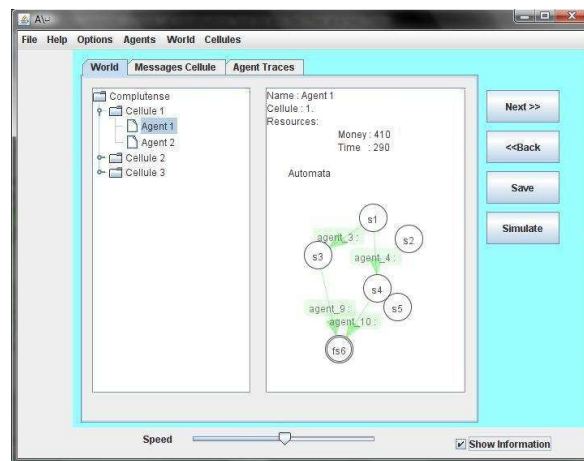
to move to a new state (and, subsequently, call the next agent to perform the new task). We try to let it solve a certain task. This phase is presented in Figure 1.8.

First $agent_1$ and $agent_2$ send a message. Both of them need to move from state 1 to state 6, but $agent_1$ owns more money than $agent_2$ while it has less time to arise it. So, we expect that $agent_1$ will find a faster, although more expensive, way to proceed than $agent_2$.

We will show the decision that $agent_1$ takes in order to obtain a faster path. The $agent_2$ behaves in the same way as $agent_1$. The first message from $agent_1$ has type=1, that is, information, and it is a broadcast message to another communication cellule. This message is used to look for all agents in that cellule that have the same final state. This first message is sent from C_1 to C_2 .

The agents $agent_8$, $agent_9$, $agent_{10}$ have their goal as s_6 . All of them answer to the previous message by sending information concerning their amounts of *resources*. $agent_1$ starts building a tree in order to decide the best path to the goal.

Now $agent_1$ asks, by sending a broadcast message to C_2 , who has as goal state s_2 , s_3 , s_4 . These messages flow through C_2 . We have that $agent_4$, $agent_6$, and $agent_3$ will answer with messages sending s_1 , s_1 and s_2 . Because the initial state of $agent_1$ is s_1 , it only has to

Fig. 1.8. Phase 2 in $\mathcal{A}\backslash\sqcup$.Fig. 1.9. Phase 3 in $\mathcal{A}\backslash\sqcup$.

obtain one possible way for s_2 .

In the final step of this phase agent₁ sends to C_2 a message asking for agents whose goal is s_2 . No agents will answer to it. The $\mathcal{A}\backslash\sqcup$ tool has a timeout module which sends an internal transition for agent₁ denoting that nobody is going to answer this last message. When this timeout is sent, agent₁ starts to decide which path it prefers to follow its utility function.

In phase 3 we have the situation described in Figure 1.9. First, agent₁ has all the possible paths to achieve its goal already displayed in front of him. By applying its utility

function, this agent decides that the best way is to step through s_3 . So, it calls agent₃ to start its internal processes. Then, agent₃ will then restart the whole process until it finally calls agent₁₁ which is an atomic agent. Therefore, it will start to transform the resources by taking part of the money (in s_1) and transforming it into land (reaching s_2).

1.6. Conclusions and future work

In this paper we have presented a formalism to represent complex hierarchical systems where tasks can be distributed and/or *subcontracted* among agents. We are aware that our formalism is difficult to use since there are a lot of mathematical machinery underlying the definition of our systems. Thus, we have decided to build a tool that fully implements our methodology. In this way, a user of our methodology does not need to pay attention to the formal details and can concentrate on defining the appropriate hierarchical structure.

The system we have implemented allows a more direct approach to model systems that will expand with the use of it by the user. Agent technology allows to have an unbounded growth of the system, and having declared all the possible atomic task that a system can perform as outputs will allow the system to complete any petition that the user can foresee.

There are of course limitations, due to the nature of the simplified perceived world as a vector of resources. In this line a possible future implementation should maybe base the world representation in a BDI (belief, desire, intention) system, with a modal logic and allowing symbolic representation of the world and user needs. But until date we have considered it outside of the focus of our research, to be able to advances in other areas.

There are at least two lines for future work. On the one hand, there is a lot of room to continue the theoretical study. In particular, we can exploit the trace relation between agents so that we can define a *conformance relation* to determine whether a real system correctly implements one of our worlds. On the other hand, more practical, we have used our tool only with small/medium size examples. We are working on the complete definition of a *real* system by using our tool. Specifically, we are considering²⁸ as a non-trivial system to be described in our tool.

References

1. R. Guttman, A. Moukas, and P. Maes, Agent-mediated electronic commerce: A survey, *The Knowledge Engineering Review*. **13**(2), 147–159, (1998).
2. L. Rasmusson and S. Janson, Agents, self-interest and electronic markets, *The Knowledge Engineering Review*. **14**(2), 143–150, (1999).
3. T. Eymann. Markets without makers - a framework for decentralized economic coordination in multiagent systems. In *2nd Int. Workshop on EElectronic COMmerce, WELCOM'01, LNCS 2232*, pp. 63–74. Springer, (2001).
4. M. Dastani, N. Jacobs, C. Jonker, and J. Treur. Modelling user preferences and mediating agents in electronic commerce. In *Agent Mediated Electronic Commerce, The European AgentLink Perspective, LNCS 1991*, pp. 163–193. Springer, (2001).
5. J. Lang, L. v. Torre, and E. Weydert, Utilitarian desires, *Autonomous Agents and Multi-Agent Systems*. **5**(3), 329–363, (2002).

6. M. McGeachie and J. Doyle. Utility functions for ceteris paribus preferences. In *AAAI Workshop on Preferences in AI and CP: Symbolic Approaches*, pp. 33–38. AAAI Press, (2002).
7. J. Keppens and Q. Shen. A calculus of partially ordered preferences for compositional modelling and configuration. In *AAAI Workshop on Preferences in AI and CP: Symbolic Approaches*, pp. 39–46. AAAI Press, (2002).
8. F. Bacchus and A. Grove. Graphical models for preference and utility. In *Uncertainty in Artificial Intelligence, UAI'95*, pp. 3–10. Morgan Kaufmann, (1995).
9. T. Sandholm. Agents in electronic commerce: Component technologies for automated negotiation and coalition formation. In *2nd Int. Workshop on Cooperative Information Agents, CIA'98, LNCS 1435*, pp. 113–134. Springer, (1998).
10. S. Kraus, Negotiation and cooperation in multi-agent systems, *Artificial Intelligence*. **94**(1-2), 79–98, (1997).
11. A. Lomuscio, M. Wooldridge, and N. Jennings. A classification scheme for negotiation in electronic commerce. In *Agent Mediated Electronic Commerce, The European AgentLink Perspective, LNCS 1991*, pp. 19–33. Springer, (2001).
12. C. Hoare, *Communicating Sequential Processes*. (Prentice Hall, 1985).
13. R. Milner, *Communication and Concurrency*. (Prentice Hall, 1989).
14. J. Bergstra, A. Ponse, and S. Smolka, Eds., *Handbook of Process Algebra*. (North Holland, 2001).
15. W. Brauer, W. Reisig, and G. Rozenberg, Eds., *Petri Nets I: Central Models and Their Properties, LNCS 254*. (Springer, 1987).
16. W. Brauer, W. Reisig, and G. Rozenberg, Eds., *Petri Nets II: Applications and Relationships to Other Models of Concurrency, LNCS 255*. (Springer, 1987).
17. I. Lomazova, Communities of interacting automata for modelling distributed systems with dynamic structure, *Fundamenta Informaticae*. **60**(1-4), 225–235, (2004).
18. I. Lomazova. Nested Petri Nets for adaptive process modeling. In *Pillars of Computer Science, Essays Dedicated to Boris Trakhtenbrot on the Occasion of His 85th Birthday, LNCS 4800*, pp. 460–474. Springer, (2008).
19. M. Núñez and I. Rodríguez. PAMR: A process algebra for the management of resources in concurrent systems. In *21st IFIP WG 6.1 Int. Conf. on Formal Techniques for Networked and Distributed Systems, FORTE'01*, pp. 169–185. Kluwer Academic Publishers, (2001).
20. M. Núñez, I. Rodríguez, and F. Rubio, Formal specification of multi-agent e-barter systems, *Science of Computer Programming*. **57**(2), 187–216, (2005).
21. M. Núñez, I. Rodríguez, and F. Rubio, Specification and testing of autonomous agents in e-commerce systems, *Software Testing, Verification and Reliability*. **15**(4), 211–233, (2005).
22. M. Merayo, M. Núñez, and I. Rodríguez. Formal specification of multi-agent systems by using EUSMs. In *2nd IPM Int. Symposium on Fundamentals of Software Engineering, FSEN'07, LNCS 4767*, pp. 318–333. Springer, (2007).
23. P. Maes. The dynamics of action selection. In *11th International Joint Conference on Artificial Intelligence IJCAI'89*, pp. 991–997. Morgan Kaufmann, (1989).
24. Y. Shoham, Agent-oriented programming, *Artificial Intelligence*. **60**(1), 51–92, (1993).
25. B. Geisler, V. Ha, and P. Haddawy. Modeling user preferences via theory refinement. In *5th Int. Conf. on Intelligent User Interfaces, IUI'01*, pp. 87–90. ACM Press, (2001).
26. V. Ha and P. Haddawy, Similarity of personal preferences: Theoretical foundations and empirical analysis, *Artificial Intelligence*. **146**(2), 149–173, (2003).
27. A. Mas-Colell, M. Whinston, and J. Green, *Microeconomic Theory*. (Oxford University Press, 1995).
28. C. Andrés, M. Merayo, and M. Núñez. Formal development of a complex information system. In *3rd Int. Conf. on Systems, ICONS'08*, pp. 118–123. IEEE Computer Society Press, (2008).